Université de Montpellier — Faculté des Sciences

HAI913I — Évolution et restructuration des logiciels

TP 1 — Analyse statique

Compte-rendu

Enseignant responsable : Bachar Rima
Année universitaire : 2026–2027
Version de travail du 11 octobre 2026

Alexandre SIREIX

Numéro étudiant : 22414718
Parcours : Génie logiciel (GL)

Code source du projet

`https://github.com/ASireix/HAI913ITP1`

Vidéo de présentation

`https://www.youtube.com/watch?v=Zs6KZkgmhW4`

Table des matières

1 Objectifs et périmètre **2**

2 Conception générale et architecture **2**

3 Réalisation étape par étape **2**
3.1 A0 — Configuration et analyse des sources 2
3.2 A1 — Comprendre et parcourir l'AST 3
3.3 A2 — Extraire la structure 3
3.4 A3 — Extraire les appels et leurs receveurs 3
3.5 B1 — Modèle de faits 4
3.6 B2 — Métriques 5
3.7 B3 — Graphe d'appel 6
3.8 B4 — Interface et robustesse 6

4 Validation et résultats observés **7**

5 Difficultés et état d'avancement **8**

6 Utilisation d'outils d'intelligence artificielle **8**

7 Regard critique **8**

8 Conclusion **8**

1 Objectifs et périmètre

L'objectif est de construire un analyseur statique de projets Java : extraire leur structure et leurs appels, puis exploiter ces faits pour calculer des métriques et construire un graphe d'appel. Le programme analysé n'est pas exécuté. L'analyse repose sur Eclipse JDT, à partir du squelette fourni avec le TP.

La version décrite ici réalise les étapes A0 à B4 : extraction, stockage des faits, métriques, graphe et interface en ligne de commande. Elle reconnaît les classes, interfaces, énumérations et records nommés, de premier niveau ou membres. Les classes locales, anonymes et types d'annotation sont exclus du comptage des types. Les appels figurant dans une classe anonyme ou une lambda située dans le corps d'une méthode analysée restent parcourus.

Les constantes d'énumération, composants de records et méthodes implicites ne sont pas comptés comme membres. Les appels retenus sont les appels ordinaires et les appels de méthodes via `super`, conformément aux conventions du sujet. Les références de méthodes et appels de constructeurs ne sont pas ajoutés aux appels A3.

2 Conception générale et architecture

La chaîne de traitement sépare la lecture des sources, l'extraction des faits et leur présentation. Cette séparation suit les principes de l'annexe Workflow [**?**], sans reprendre obligatoirement son organisation de classes.

Chemin du projet
↓
`ProjectSources` : racine et fichiers Java
↓
`JdtParser` : AST et bindings
↓
`JdtVisitorClasses` et `JdtVisitorMethods`
↓
`ProjectFacts` et `AnalysisFacts` : données sans JDT
↓
`MetricsService` : calcul des métriques B2
↓
`ConsoleReporter` : structure, appels, totaux et métriques

`Main` orchestre ces étapes. Le visiteur A1 conserve un affichage direct, car son rôle est précisément d'observer l'AST. Pour A2 et A3, les visiteurs retournent des données et ne réalisent plus la présentation. Les fonctions autrefois regroupées dans `NodeUtils` ont ainsi été réparties entre extraction et affichage.

Le modèle actuel utilise des records Java : un `TypeFact` contient des `FieldFact` et des `MethodFact` ; chaque méthode contient ses `CallFact`. Les listes sont copiées avec `List.copyOf`. Le module de présentation ne dépend d'aucune classe JDT. Le modèle complet est regroupé dans `ProjectFacts`. Les cibles textuelles simplifiées sont conservées pour A3 ; des identifiants distincts, incluant les types des paramètres, permettent de distinguer les surcharges dans les données.

3 Réalisation étape par étape

3.1 A0 — Configuration et analyse des sources

Le projet utilise Maven, JDT Core 3.46.0 et un niveau de langage Java 17. Les vérifications ont été réalisées avec le JDK 21 installé dans `/usr/lib/jvm/java-21-openjdk`. Le parser utilise

`AST.getJLSLatest()` pour le niveau d'API de l'AST, distinct du niveau de langage du code analysé.

La configuration active `setResolveBindings(true)` et `setBindingsRecovery(true)`. L'environnement fournit la racine des sources, l'encodage UTF-8, les éventuels JAR et le JDK courant. Les fichiers sont analysés en un lot via `createASTs`. Les diagnostics sont lus dans chaque `CompilationUnit`.

Le projet de validation produit 14 unités de compilation et aucune erreur. Le test fourni `ParserSmokeTest` vérifie également la résolution d'un appel entre deux fichiers.

Une difficulté initiale provenait du Java utilisé par Maven : le terminal sélectionnait Java 11 et signalait l'absence de compilateur. Forcer `JAVA_HOME` vers le JDK 21 a permis de compiler. La version du Java exécutant Maven doit donc être vérifiée séparément de la présence de `javac`.

3.2 A1 — Comprendre et parcourir l'AST

Le parser construit l'arbre ; `ASTVisitor` définit les traitements appliqués pendant son parcours. `preVisit2` et `postVisit` permettent de suivre la profondeur. Le mode complet affiche les types de tous les nœuds. Le mode `lite` conserve une vue réduite des classes, champs, méthodes et appels au constructeur parent.

Pour `Dvd.java`, les éléments repérés sont un `TypeDeclaration`, un `FieldDeclaration` et trois `MethodDeclaration`, dont un constructeur contenant un `SuperConstructorInvocation`. Pour `Category.java`, la déclaration du type est un `EnumDeclaration` : elle est visible dans le parcours complet, mais n'est pas reçue par `visit(TypeDeclaration)`.

Renvoyer `true` depuis `visit` autorise la visite des enfants ; renvoyer `false` élague ces enfants sans interrompre le reste de l'arbre. Les opérations `endVisit` et `postVisit` restent exécutées. `preVisit2` intervient avant la visite spécialisée : son retour faux évite également cette visite spécialisée [**?**].

3.3 A2 — Extraire la structure

Chaque déclaration de type acceptée déclenche l'extraction de son identité, de son genre, de son paquetage et de ses membres directs. La liste `bodyDeclarations()` évite d'attribuer à `LoanService` les membres de son type membre `Loan`. Le parcours récursif permet ensuite de traiter ce dernier séparément.

Chaque fragment d'un `FieldDeclaration` correspond à un attribut : `int a, b;` en produit deux. Le type déclaré, les dimensions supplémentaires et la visibilité sont conservés. Sans modificateur de visibilité, un champ de classe est de visibilité package ; un champ d'interface est public implicite. Les méthodes sont décrites par leur nom, leur nombre de paramètres et `isConstructor()`.

La chaîne d'héritage provient des bindings :

```
for (ITypeBinding parent = binding.getSuperclass();
     parent != null
         && !parent.getQualifiedName().equals("java.lang.Object");
     parent = parent.getSuperclass()) {
    parents.add(parent.getQualifiedName());
}
```

Les interfaces directes sont obtenues séparément par `getInterfaces()`. Une interface n'a pas de superclasse ; ses interfaces parentes figurent dans cette liste.

3.4 A3 — Extraire les appels et leurs receveurs

Un visiteur spécialisé est lancé sur le corps de chaque méthode retenue, lorsqu'il existe. Cela exclut les initialiseurs de champs. Il collecte les `MethodInvocation` et `SuperMethodInvocation`,

sans couper le parcours des lambdas et classes anonymes. La méthode propriétaire du corps conserve les appels collectés.

La cible vient de `resolveMethodBinding()`, puis `getMethodDeclaration()`, afin de retrouver la déclaration d'origine. La ligne est obtenue par `unit.getLineNumber(node.getStartPosition())`. Les deux visites partagent une fonction de collecte ; la présentation est centralisée dans `ConsoleReporter`.

Forme de l'appel	Type statique du receveur
`objet.m()`	Binding du type de l'expression receveuse.
`m()` statique	Type déclarant la méthode résolue.
`m()` d'instance	Type englobant retrouvé en remontant les parents AST.
`super.m()`	Superclasse directe du type englobant.

Le type du receveur est converti par `getErasure().getQualifiedName()`. Il ne faut pas le confondre avec le type déclarant la cible : `zadig.getTitle()` a pour receveur `Book`, tandis que la méthode est déclarée dans `Item`.

L'expression receveuse peut être absente et son binding peut être nul ; le binding de méthode peut également être nul. Ces cas sont traités séparément. Sur une dépendance absente, l'appel est conservé comme non résolu et l'analyse continue. Le caractère interne est actuellement déterminé par `isFromSource()` sur le type déclarant la cible.

3.5 B1 — Modèle de faits

`FactExtractor` constitue la frontière entre JDT et le reste du programme. À partir des AST déjà construits, il rassemble les résultats des visiteurs dans un `ProjectFacts`. Celui-ci contient les fichiers avec leur chemin relatif et leur nombre de lignes physiques, les types et les diagnostics. Les types contiennent leurs attributs et méthodes, et les méthodes leurs appels. Aucune classe du paquetage `model` ne dépend de JDT.

Identité des méthodes. Un nom et un nombre de paramètres ne suffisent pas à distinguer les surcharges. `MethodIdentity` applique donc la même convention aux déclarations et aux cibles : type qualifié, nom et types des paramètres. Par exemple :

```
library.service.Catalog#add(library.model.Item)
library.service.Catalog#add(library.model.Item,library.model.Item)
```

Les constructeurs utilisent le nom interne `<init>`. Les bindings sont ramenés à leur déclaration d'origine et les types des paramètres sont effacés pour obtenir une identité commune aux déclarations génériques et à leurs utilisations. Sans binding de déclaration, les types écrits dans les paramètres servent de repli ; cette identité dégradée n'offre pas les mêmes garanties que la résolution sémantique.

Liens des appels. Chaque `CallFact` conserve `callerId` et un `Optional<String>` pour `targetId`. L'absence de cible représente un appel non résolu, qui reste compté. Les champs déjà utilisés pour l'affichage A3 sont conservés : les identifiants complets n'imposent pas un affichage plus détaillé. `methodsById()` construit un index des méthodes et refuse les identifiants dupliqués.

Informations préparant B2. Chaque méthode conserve les types des paramètres et les lignes physiques de son corps, accolades incluses, calculées à partir des positions JDT. Une méthode sans corps a une longueur nulle. Les lignes des fichiers sont comptées avec `Files.readAllLines`, commentaires et lignes vides inclus.

Stabilité et validation. Les fichiers sont triés par chemin, les types par nom et les méthodes par identifiant. Les appels gardent l'ordre du parcours source ; les listes exposées ne sont pas modifiables. `ProjectFactsTest` compare les modèles de deux analyses indépendantes, vérifie que les surcharges sont distinctes et que les identifiants des appelantes et des cibles internes désignent des méthodes présentes dans le modèle. Il vérifie également la conservation d'un appel non résolu sans cible inventée. Les totaux A2 et A3 restent identiques après cette transformation.

3.6 B2 — Métriques

`MetricsService` reçoit uniquement un `ProjectFacts` et le seuil X. Il retourne un `MetricsResult`, que `ConsoleReporter` met en forme. Le calcul ne relit pas les fichiers, ne reparcourt pas les AST et ne dépend pas de JDT.

Les questions Q1 à Q4 comptent les types, les lignes des fichiers, les méthodes et les noms de paquetages distincts. Q5 et Q7 divisent les totaux des méthodes et attributs par le nombre de types. Q6 divise la somme des lignes des corps par le nombre de méthodes ayant un corps : les méthodes abstraites n'abaissent donc pas artificiellement cette moyenne. Les calculs conservent leur précision ; seul l'affichage est arrondi à deux décimales. Par convention d'implémentation, une moyenne sur un ensemble vide vaut zéro.

Pour Q8, Q9 et Q12, la fonction commune `topTenPercent` trie les candidats par valeur décroissante, puis par nom pour départager l'affichage des ex æquo. Elle calcule $k = \lceil n/10 \rceil$ sur tous les candidats. La valeur au rang k devient le seuil ; tous les candidats de valeur positive supérieure ou égale à ce seuil sont conservés. Q12 effectue ce calcul séparément pour les méthodes de chaque type. Un ensemble vide ou composé uniquement de valeurs nulles donne une liste vide.

```
int k = (candidates.size() + 9) / 10;
int cutoff = sorted.get(k - 1).value();
return sorted.stream()
    .filter(r -> r.value() > 0 && r.value() >= cutoff)
    .toList();
```

Cet extrait intervient apres le traitement du cas vide. Q10 prend l'intersection des classements Q8 et Q9. Q11 applique une comparaison strictement supérieure à X. Q13 conserve toutes les méthodes atteignant le maximum du nombre de paramètres ; sur un projet sans méthode, le maximum vaut zéro et la liste est vide.

Question	Attendu	Obtenu
Q1–Q4	15 types ; 493 lignes ; 59 méthodes ; 4 paquetages	Identique
Q5–Q7	3,93 ; 4,64 ; 1,40	Identique ; Q6 : 246 lignes / 53 corps
Q8	Item (10), Member (6), LoanService (6)	Identique ; 3 retenus pour $k = 2$
Q9–Q10	Item et LoanService	Identique ; 5 attributs chacune
Q11	X=5 : 3 types ; X=4 : 6 types	Identique
Q12	Item : checkOut (7) ; LoanService : borrow (14)	Identique
Q12, ex æquo	Member : ses 6 méthodes (3 lignes) ; Loanable : aucune	Identique
Q13	5 paramètres : Statistics#report	Identique

`MetricsTest` vérifie ces résultats, le seuil strict, l'arrondi supérieur, les ex æquo, l'exclusion des zéros et les moyennes sur un modèle vide. Un seuil négatif est refusé. Les résultats sont affichés avec la locale française, notamment la virgule décimale.

3.7 B3 — Graphe d'appel

`CallGraphService` construit un graphe orienté à partir de `ProjectFacts`, sans utiliser JDT. Chaque méthode comptée est un nœud, même si elle n'appelle aucune méthode et n'a aucune appelante. Les constructeurs sont donc présents, mais les appels de constructeurs ne sont pas ajoutés : l'option facultative du sujet n'est pas implémentée.

Les appels résolus dont la cible figure dans l'index des méthodes du modèle donnent des arcs. Une paire d'identifiants (appelante, appelée) sert de clé ; chaque occurrence incrémente son poids. Une récursion produit une boucle sur le même nœud. Les cibles absentes du graphe sont comptées à part, de même que les appels non résolus, sans création de nœuds artificiels. Sur le projet de validation, ces cibles absentes sont les méthodes externes attendues.

```
weights.merge(new Pair(call.callerId(), target), 1, Integer::sum);
```

Le graphe `CallGraph` expose les nœuds et arcs sous forme de collections non modifiables, triées par identifiants. Il propose `callees(id)` pour les arcs sortants et `callers(id)` pour les arcs entrants. Ces recherches filtrent la liste des arcs ; ce choix simple est suffisant à l'échelle du TP. Une méthode inconnue est distinguée d'une méthode connue sans voisin.

La sortie texte de `ConsoleReporter` liste les méthodes, puis les arcs sous la forme `appelante -> appelee [poids=n]`, avec les compteurs globaux. Ce format est lisible directement, déterministe et redirigeable dans un fichier. Il évite une dépendance graphique sans perdre les nœuds isolés. Les options `-callers` et `-callees` permettent les deux recherches à partir d'un identifiant complet entre guillemets ; elles peuvent être combinées.

Mesure	Attendu	Obtenu
Nœuds	59	59
Arcs	44	44
Sites internes (somme des poids)	46	46
Appels externes	48	48
Appels non résolus	0	0

`CallGraphTest` vérifie ces nombres et les exemples du sujet : l'arc de `Catalog#add(Item,Item)` vers `Catalog#add(Item)` a un poids de 2 ; `TextUtils#repeat` possède un arc vers elle-même. La seule appelante de `Loanable#checkOut(Member)` est `LoanService#borrow`, tandis que `Item#checkOut(Member)` n'a aucune appelante. Il n'y a pas d'expansion des implémentations possibles d'un appel virtuel.

Les tests vérifient aussi le graphe vide, une recherche inconnue, la stabilité entre deux analyses et le maintien du compteur des appels non résolus sur `missing-dep`. Une exécution de la ligne de commande a confirmé les résultats des recherches dans les deux sens.

3.8 B4 — Interface et robustesse

`CliOptions` sépare la lecture des arguments de l'orchestration dans `Main`. L'option `-action` sélectionne `summary`, `ast`, `structure`, `calls`, `metrics`, `graph` ou `all` (défaut). Un bilan des fichiers et diagnostics est toujours affiché. L'action AST explicite parcourt tous les fichiers ; le mode complet conserve la démonstration A1 sur Dvd et Category. `-lite` ou `lite` réduit l'affichage AST.

Le seuil `-threshold X` est un entier positif ou nul, avec une valeur par défaut de 5 si l'option est omise. Une option sans valeur, un seuil mal formé ou négatif, une action inconnue ou une recherche hors des actions graph/all sont refusés. `-help` décrit les options. Les identifiants des recherches et les chemins contenant des espaces doivent être placés entre guillemets.

Les codes de sortie sont documentés : 0 pour une analyse terminée, même partielle avec diagnostics ; 2 pour les arguments invalides ; 3 pour une entrée inexploitable ou une méthode inconnue ; 4 pour une autre exception empêchant l'analyse. Les exceptions interceptées sont traduites en messages explicites, sans pile d'exception brute.

Cas vérifié	Résultat observé	Code
Dossier inexistant	Message indiquant que le chemin n'est pas un répertoire	3
`no-java-files`	Message signalant l'absence de fichier Java	3
X = abc ou valeur absente	Refus et message d'usage	2
Méthode inconnue	Message avec l'identifiant demandé	3
`syntax-error`	Diagnostics pour Broken ; Helper est analysé	0
`missing-dep`	Appel non résolu conservé ; analyse poursuivie	0

`CliTest` vérifie les options et plusieurs erreurs de bout en bout. Les codes ont également été contrôlés par exécution du JAR dans des processus séparés. Une copie des sources et des jeux de tests, sans `target/`, a été placée dans un chemin contenant des espaces. La commande `mvn -o clean package` y a reconstruit le JAR et exécuté les tests ; une analyse du projet de validation dans ce même chemin a retrouvé le bilan B3. Le mode hors ligne utilise les dépendances déjà présentes dans le cache Maven : il ne simule pas un premier téléchargement avec un cache vide.

L'environnement vérifié est Arch Linux x86_64, OpenJDK 21.0.12.1, Maven 3.9.16 et JDT Core 3.46.0. Le README fournit la commande standard `mvn clean package`, les exemples d'actions et les remèdes aux problèmes Java. Les jeux de validation sont placés sous `source/resources`, car les tests les utilisent.

4 Validation et résultats observés

Les résultats suivants ont été vérifiés sur le projet fourni, après séparation de l'extraction et de l'affichage.

Mesure	Attendu	Obtenu
Unités de compilation	14	14
Erreurs sur validation	0	0
Types	15	15
Classes / interfaces / énumérations	12 / 2 / 1	12 / 2 / 1
Attributs	21	21
Méthodes / constructeurs	59 / 9	59 / 9
Appels ordinaires / via super	93 / 1	93 / 1
Appels internes / externes / non résolus	46 / 48 / 0	46 / 48 / 0

Les vérifications ciblées donnent 6 méthodes et 5 attributs pour `LoanService`, contre 4 méthodes et 3 attributs pour `LoanService.Loan`. La chaîne de `LoanException` est `java.lang.Exception`, puis `java.lang.Throwable`. Les visibilités des cinq attributs de `Item` correspondent au sujet.

Pour A3, les receveurs de `loanable.checkOut(member)`, `zadig.getTitle()` et `super.describe()` sont respectivement `Loanable`, `Book` et `Item`. Sur `robustness/missing-dep`, deux erreurs de compilation sont signalées, mais les deux appels sont conservés : `capitalize` est non résolu et `decorate` est résolu et interne.

`PartAReportTest` automatise les totaux, l'appartenance des membres de `LoanService`, certains receveurs et la conservation des appels non résolus. Il complète `ParserSmokeTest`. Les dossiers `resources/validation` et `resources/robustness`, présents dans `source`, sont utilisés par ces tests.

Commandes exécutables depuis `source` :

```
JAVA_HOME=/usr/lib/jvm/java-21-openjdk mvn package
java -jar target/hai913i-tp1-analyzer.jar resources/validation lite --threshold 5
java -jar target/hai913i-tp1-analyzer.jar resources/robustness/missing-dep lite
```

Les vérifications de robustesse et de reconstruction sont détaillées dans la section B4.

5 Difficultés et état d'avancement

Trois difficultés ont guidé l'implémentation : distinguer construction et parcours de l'AST ; séparer type du receveur et type déclarant la cible ; comprendre qu'un binding est un objet sémantique et non un simple nom à afficher. L'affichage direct d'un binding produisait une description très longue ; `getQualifiedName()` fournit le nom utile.

Le parcours des types imbriqués exige également de distinguer récursion du visiteur et lecture des membres directs. Enfin, la séparation des faits et de la présentation a nécessité de déplacer des fonctions initialement situées dans `NodeUtils`.

À cette date, A0–B4 sont vérifiés sur les points de contrôle présentés. Les cibles affichées sous la forme `Type#methode()` restent distinctes des identifiants complets désormais stockés pour le graphe.

6 Utilisation d'outils d'intelligence artificielle

J'ai utilisé OpenAI Codex avec le modèle ChatGPT Astra 6, réglé sur le niveau de raisonnement *low*, pour expliquer des notions, aider à la réalisation d'une partie du code et rédiger le rapport.

Mes demandes ont notamment porté sur le rôle du parser et de l'AST, la différence entre parcours récursif et lecture de `bodyDeclarations()`, la gestion des bindings nuls et la recherche du type statique du receveur. Lorsque les premières explications ne suffisaient pas, j'ai demandé des indices supplémentaires, par exemple pour remonter jusqu'au type englobant d'un appel implicite.

L'assistant a aussi proposé des refactorisations, réalisé le module d'affichage et commencé l'implémentation de B2. J'ai regardé et exécuté le code de mon côté, et je le réécris lorsque c'est nécessaire. Les résultats ont été comparés aux points de contrôle du sujet. Je reste responsable du code rendu et des explications présentées dans ce rapport.

7 Regard critique

La cible résolue par JDT est celle déterminée à la compilation. Elle ne garantit pas quelle redéfinition sera exécutée à l'exécution. Le futur graphe ne modélisera pas automatiquement la réflexion, les proxys ou l'injection de dépendances. Une dépendance absente limite la résolution, mais ne doit pas faire disparaître silencieusement un appel.

Les points de contrôle sont nécessaires, mais ne démontrent pas seuls le fonctionnement sur toutes les constructions Java. Les tests actuels restent ciblés ; des cas supplémentaires pour les types génériques, les receveurs anonymes et les interactions entre génériques et surcharges renforceront la validation. Le graphe utilise des recherches linéaires dans la liste des arcs ; des index d'adjacence seraient utiles pour des projets plus grands. Les métriques fournissent des indicateurs de taille ou de concentration, pas une mesure autonome de la qualité d'un programme.

8 Conclusion

La partie A fournit une extraction vérifiée de la structure et des appels du projet de validation. La séparation entre visiteurs, données et affichage prépare l'exploitation de ces résultats. B1 fournit maintenant un modèle indépendant de JDT avec identifiants de méthodes et liens d'appels ; B2 retrouve les résultats attendus des treize métriques. B3 construit le graphe pondéré et permet les recherches d'appelantes et d'appelées. B4 permet de sélectionner les sorties et fournit des messages explicites pour les erreurs ; les tests de robustesse et la reconstruction sur une copie propre ont été effectués.